

Simulador de Execução Dinâmica de Processos com MLFQ

Manual do usuário e documentação técnica



Integrantes: Lucca Marques e Rodrigo Gagliardi

Professor: Fabiano Passuelo Hessel

Data: 24/09/2026

Sumário

1 Visão geral

2 Requisitos e compilação

3 Como executar

4 Arquivo de configuração

5 Formato dos programas

6 Regras de escalonamento

7 Como ler a saída

8 Decisões de interpretação

9 Exemplo do enunciado e divergência com o gabarito do PDF

10 Arquitetura do código

11 Testes e validação

12 Limitações conhecidas

1. Visão geral

O simulador reproduz uma máquina baseada em acumulador que executa vários programas ao mesmo tempo, escalonados por um **MLFQ** (Multilevel Feedback Queue) de duas filas. O tempo avança em **ticks**: cada tick é uma unidade de tempo (UT), e em cada tick no máximo uma instrução é executada.

O simulador faz três coisas:

1. **Lê programas** escritos em um assembly simplificado (arquivos `.asm`). Erros de sintaxe são detectados na leitura, antes de qualquer execução, com o arquivo e a linha.
2. **Executa as instruções**, uma por UT. Cada processo tem seu próprio acumulador, contador de programa e memória; dois processos que rodam o mesmo programa não interferem um no outro.
3. **Escalona os processos** entre a CPU, as duas filas de prontos e a lista de bloqueados, e ao final mostra o diagrama de Gantt e as estatísticas de turnaround e espera.

A entrada é um **arquivo de configuração** que lista os processos (nome, instante de chegada, prioridade e programa). A saída mostra, a cada UT, o estado de cada processo, o conteúdo das filas e dos bloqueados, e as mensagens dos programas.

Estrutura de pastas

T1_Sisop/	
build.bat, build.sh	compilam do zero e executam
src/	código-fonte Java
programas/	programas .asm
configs/	arquivos de configuração
docs/manual.html	fonte deste manual
README.md	resumo do projeto

2. Requisitos e compilação

- **Java (JDK) 8 ou superior.** Testado com JDK 8, 21 e 23. Não há bibliotecas externas nem ferramentas de build (Maven/Gradle).
- **Windows** (cmd ou PowerShell) ou **Linux**.

Para conferir se o JDK está instalado: `javac -version`.

Compilar pelo script (recomendado)

Os scripts apagam a pasta `bin/`, compilam todo o projeto do zero e já executam a simulação com os argumentos recebidos.

Terminal	Comando
cmd (Windows)	<code>build.bat configs\cenario_c.txt</code>
PowerShell (Windows)	<code>.\build.bat configs\cenario_c.txt</code>
Linux	<code>./build.sh configs/cenario_c.txt</code>

Os comandos devem ser executados **na pasta do projeto**. No Windows, uma forma prática é abrir a pasta no Explorador de Arquivos, digitar `powershell` na barra de endereço e pressionar Enter.

Compilar manualmente

Na pasta do projeto, compile **todos** os arquivos de uma vez:

```
javac -encoding UTF-8 -d bin src/*.java src/**/*.java
java -cp bin Main configs/cenario_c.txt
```

Atenção: compilar um arquivo isolado (por exemplo, `javac src/simulation/Scheduler.java`) falha com *"package assembly does not exist"*, porque o compilador não encontra as classes dos outros pacotes. Use o script ou o comando acima.

Os scripts configuram a saída em UTF-8 para os acentos aparecerem corretamente. No Windows, `build.bat` executa `chcp 65001`, o que muda a codificação daquela janela do terminal até ela ser fechada.

3. Como executar

Formato geral: `build.bat [configuração] [modo]`. Sem argumentos, usa `configs/cenario_c.txt` no modo tabela.

Modos de exibição

Segundo argumento	Comportamento
<code>(nenhum)</code>	Modo tabela. Imprime uma linha por tick, depois o Gantt e as estatísticas.
<code>passo</code>	Modo apresentação. Limpa a tela e mostra um tick por vez; avança a cada Enter.
um número, ex. <code>800</code>	Modo apresentação automático. Avança sozinho a cada 800 ms.

Os modos só mudam a exibição: a simulação e os resultados são sempre os mesmos.

```
.\build.bat configs\cenario_c.txt      (tabela)
.\build.bat configs\cenario_c.txt passo (um tick por Enter)
.\build.bat configs\cenario_c.txt 800   (automático, 0,8 s por tick)
```

O modo apresentação precisa de uma janela de terminal interativa (cmd, PowerShell, terminal do Linux ou terminal integrado do VS Code). Pelo botão "Run" de uma IDE, que não passa argumentos, o programa roda no modo tabela.

Configurações incluídas

Arquivo	Conteúdo
<code>configs/cenario_a.txt</code>	P01 sozinho (teste1.asm)
<code>configs/cenario_b.txt</code>	P02 sozinho (teste2.asm)
<code>configs/cenario_c.txt</code>	Exemplo do enunciado: P01 e P02 juntos
<code>configs/caso1.txt</code>	Caso de teste de instruções; pede um número pelo teclado (use <code>-7</code>)
<code>configs/caso2.txt</code>	Caso de teste de escalonamento com três processos

Entrada pelo teclado (SYSCALL 2)

Quando um programa executa `SYSCALL 2`, o simulador pede um inteiro e repete o pedido até receber um valor válido. Para testes reproduzíveis, a entrada pode vir de um arquivo (no cmd ou no Linux):

```
build.bat configs\caso1.txt < configs\caso1_entrada.txt
./build.sh configs/caso1.txt < configs/caso1_entrada.txt
```

Se a entrada terminar durante uma `SYSCALL 2`, o processo é encerrado com erro e a simulação continua.

4. Arquivo de configuração

Um processo por linha, com quatro campos separados por espaços: `nome chegada prioridade caminho.asm`. Linhas vazias e linhas iniciadas por `#` são ignoradas.

```
# nome  chegada  prioridade  programa
P01    0         3          programas/teste1.asm
P02    1         5          programas/teste2.asm
```

Campo	Regra
nome	Identifica o processo nas mensagens e na saída. Não pode se repetir.
chegada	Inteiro ≥ 0 : tick em que o processo entra no sistema.
prioridade	Inteiro de 1 a 5; maior = mais prioritário (usada na Fila 1).
caminho	Caminho do <code>.asm</code> , relativo à pasta do projeto. Pode conter espaços.

Convenção de nomes: use nomes com o mesmo número de dígitos (`P01` , `P02` , ..., `P10`). Empates são resolvidos por ordem alfabética de nome, e com o mesmo número de dígitos ela coincide com a numérica (senão `P10` viria antes de `P2`).

Qualquer erro na configuração ou nos programas impede o início da simulação e é informado com o arquivo e a linha. Exemplo:

```
Erro na configuração: e6.txt, linha 1: erro no programa de P01 -> erro1.asm, linha 6: label não de
```

São detectados: formato inválido, nome repetido, chegada negativa ou não numérica, prioridade fora de 1–5, arquivo inexistente e qualquer erro de sintaxe no programa (seção 5).

5. Formato dos programas

Um programa tem uma seção de código (`.code` ... `.endcode`) e, opcionalmente, uma seção de dados (`.data` ... `.enddata`), com uma variável por linha no formato `nome valor` .

```
.code
    LOAD limite
loop:  SUB #1      # acc = acc - 1
      STORE temp
      SYSCALL 1    # imprime acc e bloqueia 3 UTs
      LOAD temp
      BRPOS loop
      SYSCALL 0    # fim
endcode
.data
    limite 3
    temp 0
enddata
```

Instruções

Instrução	Efeito	Operando
ADD op	acc = acc + op	imediato #n ou variável
SUB op	acc = acc - op	imediato ou variável
MULT op	acc = acc × op	imediato ou variável
DIV op	acc = acc ÷ op (divisão inteira)	imediato ou variável
LOAD op	acc = op	imediato ou variável
STORE op	op = acc	somente variável
BRANY label	salta para o label	label
BRPOS label	salta se acc > 0	label
BRZERO label	salta se acc = 0	label
BRNEG label	salta se acc < 0	label
SYSCALL n	0 = termina; 1 = imprime acc; 2 = lê um inteiro para acc	0, 1 ou 2

Cada instrução custa 1 UT. `SYSCALL 1` e `SYSCALL 2` também bloqueiam o processo por 3 UTs (seção 6).

Regras de escrita

- O label pode ficar sozinho na linha (`loop:`) ou antes da instrução (`ponto1: SUB #1`).
- # colado a um número (`#5` , `#-3`) é um valor imediato. # seguido de espaço ou do fim da linha começa um comentário.

- Instruções, variáveis e labels não diferenciam maiúsculas de minúsculas (`Variable` e `variable` são a mesma variável).
- Nomes de variáveis e labels: letras sem acento, dígitos e `_`, começando por letra ou `_`.
- Valores de dados e imediatos podem ser negativos.

Erros detectados na leitura

Instrução desconhecida; operando ausente ou inválido; `STORE` com imediato; `SYSCALL` diferente de 0, 1 e 2; label ou variável não declarados; label ou variável duplicados; label no fim do código sem instrução depois; conteúdo fora das seções; seção não fechada; programa sem instruções. Exemplo:

```
erro1.asm, linha 6: label não definido: inicoi
```

6. Regras de escalonamento

Fila 0 — quantum 2	Fila 1 — quantum 4
Round Robin FIFO.	Round Robin por prioridade estática, de 5 a 1.
Recebe todo processo novo e todo processo que volta de I/O.	Mesma prioridade: ordem de chegada (FIFO).
Tem sempre a vez antes da Fila 1.	Só usa a CPU quando a Fila 0 está vazia.

Transições

Situação	O que acontece
Esgotou o quantum na Fila 0	É rebaixado para o final do seu grupo de prioridade na Fila 1, com quantum 4.
Esgotou o quantum na Fila 1	Volta para o final do seu grupo de prioridade, com quantum 4.
Alguém entrou na Fila 0 enquanto um processo da Fila 1 executava	O processo da Fila 1 é interrompido imediatamente e volta ao topo do seu grupo, mantendo o quantum que restava.
<code>SYSCALL 1</code> ou <code>2</code>	Bloqueado por 3 UTs; depois volta ao final da Fila 0, com quantum 2.
<code>SYSCALL 0</code> ou erro de execução	Finalizado.

Um processo retomado continua do ponto em que parou. A troca de contexto custa 0 UT.

Ordem dos eventos em cada tick t

1. **Desbloqueio:** quem volta de I/O em t entra no final da Fila 0.
2. **Admissão:** quem chega em t entra no final da Fila 0.
3. **Preempção:** se quem está na CPU é da Fila 1 e a Fila 0 não está vazia, ele volta ao topo do seu grupo.
4. **Dispatch:** se a CPU está livre, pega o primeiro da Fila 0; senão, o primeiro da Fila 1; senão, o tick fica ocioso.
5. **Registro** do estado para o monitor.
6. **Execução** de uma instrução; contagem de CPU, espera e I/O.
7. **Pós-execução**, nesta ordem de precedência: fim do processo → bloqueio por I/O → fim do quantum.

Essa ordem fixa torna a simulação determinística: a mesma configuração produz sempre a mesma saída.

7. Como ler a saída

Saída real do modo tabela para `configs/cenario_a.txt`:

t	CPU	Instrução	P01	Fila 0	Fila 1	Bloqueados	Evento
0	P01	LOAD valor (acc=10)	Executando	-	-	-	
1	P01	ADD #5 (acc=15)	Executando	-	-	-	quantum -> Fila 1
2	P01	STORE valor (acc=15)	Executando	-	-	-	
[P01] Impressão (SYSCALL 1): 15							
3	P01	SYSCALL 1 (acc=15)	Executando	-	-	-	imprime 15
4	--	-	Bloqueado	-	-	P01->7	
5	--	-	Bloqueado	-	-	P01->7	
6	--	-	Bloqueado	-	-	P01->7	
[P01] Finalizado (SYSCALL 0)							
7	P01	SYSCALL 0 (acc=15)	Executando	-	-	-	halt, término 8

Diagrama de Gantt (-- = CPU ociosa):

```
t : 0  1  2  3  4  5  6  7
CPU: P01 P01 P01 P01 -- -- -- P01
```

Estatísticas (em UTs):

Processo	Chegada	Término	Turnaround	CPU	I/O	Espera (direta)	TA-CPU-I/O
P01	0	8	8	5	3	0	0

Turnaround médio: 8,00 Tempo médio de espera (fila de prontos): 0,00

Coluna	Significado
t	Número do tick.
CPU / Instrução	Processo que executou no tick, a instrução e o valor de acc depois dela. -- = CPU ociosa.
Uma coluna por processo	Estado durante o tick: Pronto, Executando, Bloqueado, Finalizado, ou - se ainda não chegou.
Fila 0 / Fila 1	Processos prontos em cada fila, na ordem em que sairão. Na Fila 1, nome(prioridade) .
Bloqueados	nome->t : o tick em que o processo volta a ficar pronto.
Evento	O que aconteceu ao fim do tick: rebaixamento, preempção, impressão, leitura, término.

As filas e os estados são o retrato do tick **depois** da escolha de quem executa e **antes** do evento da última coluna. O processo que está na CPU não aparece nas filas. As mensagens dos programas ([P01] Impressão (SYSCALL 1): 15) aparecem logo antes da linha do tick em que ocorreram.

Nas estatísticas, a espera é contada diretamente (ticks em que o processo esteve pronto) e conferida pela conta turnaround – CPU – I/O. Se as duas divergirem, a linha mostra um aviso.

8. Decisões de interpretação

Pontos que o enunciado não fixa e a convenção adotada em cada um:

Situação	Decisão
Relógio	O tick t representa o intervalo $[t, t+1)$; executa no máximo uma instrução.
Bloqueio de I/O	A SYSCALL executa em t (gasta 1 UT de CPU); o processo fica bloqueado em $t+1$, $t+2$ e $t+3$ e volta a pronto em $t+4$.
Impressão e leitura	Acontecem no tick em que a SYSCALL executa, não no desbloqueio.
Término	$t+1$, onde t é o tick do SYSCALL 0. Turnaround = término – chegada.
Espera	Ticks em que o processo esteve pronto, em qualquer fila, fora da CPU.
Empate no mesmo tick	Quem volta de I/O entra na Fila 0 antes de quem chega; entre iguais, ordem alfabética de nome.
Fila 0 × Fila 0	Uma chegada na Fila 0 não interrompe outro processo da Fila 0.
SYSCALL no último tick do quantum	O processo bloqueia e não é rebaixado: I/O tem precedência sobre o fim do quantum.
Erro de execução	Divisão por zero, PC fora do programa ou fim da entrada na SYSCALL 2 encerram só aquele processo, com a mensagem [Pxx] Erro de execução: Ele é tratado como um término normal (a instrução conta 1 UT, término $t+1$, entra nas médias) e aparece marcado "(erro)".
STORE em variável não declarada	Erro de sintaxe: toda variável deve estar em .data . O próprio teste2.asm do enunciado declara temp 0 mesmo sem ler temp antes de gravá-la.
Aritmética	Inteiros de 32 bits. A divisão trunca em direção a zero ($-7 \div 2 = -3$). MULT pode estourar e "dar a volta".
Parada	Quando todos os processos terminam, ou no limite de segurança de 10.000 ticks (proteção contra laço infinito).
Figura 1 do enunciado	Declara Variable e usa variable; como os nomes não diferenciam maiúsculas, o programa funciona.

9. Exemplo do enunciado e divergência com o gabarito do PDF

configs/cenario_c.txt : P01 (teste1.asm) chega em 0 com prioridade 3; P02 (teste2.asm) chega em 1 com prioridade 5. Diagrama de Gantt produzido pelo simulador (P1 = P01, P2 = P02):



Momentos principais:

- **t = 1:** P1 esgota o quantum 2 sem terminar e é rebaixado para a Fila 1.
- **t = 4:** P2 também está na Fila 1, mas executa antes de P1 por ter prioridade 5 contra 3.
- **t = 8:** os dois estão bloqueados por I/O; a CPU fica ociosa.
- **t = 11:** P1 volta do I/O pela Fila 0, passa na frente de P2 e termina.

Saída dos programas, em ordem: [P02] Impressão (SYSCALL 1): 2, [P01] Impressão (SYSCALL 1): 15, [P01] Finalizado (SYSCALL 0), [P02] Impressão (SYSCALL 1): 1, [P02] Impressão (SYSCALL 1): 0, [P02] Finalizado (SYSCALL 0).

Processo	Término	Turnaround	CPU	I/O	Espera
P01	12	12	5	3	4 (ticks 2, 3, 4, 5)
P02	29	28	17	9	2 (ticks 1, 11)

Turnaround médio: 20 UTs. Tempo médio de espera: 3 UTs.

Divergência com o gabarito do PDF

Os números acima diferem do gabarito apresentado no PDF do enunciado, de propósito. O simulador segue as regras do enunciado, e o gabarito não é consistente com os próprios programas:

- O gabarito conta 4 UTs de CPU para P1 e 8 para P2, mas os programas executam **5** e **17** instruções, e cada instrução custa 1 UT.
- Ele agrupa instruções no mesmo instante (STORE e SYSCALL 1 de P2 em t = 4) e pula o BRPOS em t = 8.
- O log coloca [P1] Finalizado depois de [P2] ... 0, contradizendo a própria linha do tempo.

Os valores impressos pelos programas (15, 2, 1 e 0) coincidem com os do gabarito.

10. Arquitetura do código

Java puro, organizado em quatro pacotes. O fluxo tem duas etapas: a **carga**, que acontece uma vez, e o **laço de ticks**.

```
Carga:          configs/*.txt -> ConfigLoader -> AssemblyParser -> Program -> PCB (um por processo)
Cada tick:      Scheduler -> InstructionExecutor -> OperatingSystem -> Monitor / LivePanel
```

Classe	Responsabilidade
<code>Main</code>	Lê os argumentos, carrega a configuração e inicia a simulação.
assembly — leitura dos programas	
<code>AssemblyParser</code>	Lê o <code>.asm</code> com uma máquina de estados de seções e valida tudo, com o número da linha.
<code>Program</code>	Resultado imutável do parse: instruções, labels e valores iniciais dos dados. Pode ser compartilhado por vários processos.
<code>Instruction</code> , <code>InstructionType</code>	Uma instrução (tipo, operando, linha) e o enum dos mnemônicos.
cpu — execução	
<code>InstructionExecutor</code>	Executa uma instrução de um PCB. Não guarda estado; é o único ponto que avança o pc.
<code>Memory</code>	Memória de dados de um processo (cópia dos valores iniciais do programa).
<code>ExecutionResult</code> , <code>ExecutionException</code>	O que fazer após a instrução (continuar, syscall) e os erros de execução.
process — processos e Fila 1	
<code>PCB</code>	Contexto do processo: pc, acc, memória, estado, fila, quantum restante e contadores.
<code>PriorityGroupQueue</code>	Fila 1: uma fila FIFO por prioridade, com inserção no final ou no topo do grupo.
simulation — escalonamento e saída	
<code>Scheduler</code>	Relógio e os 7 passos de cada tick (seção 6).
<code>OperatingSystem</code>	Trata as syscalls (imprimir, ler, bloquear, encerrar) e os erros de execução.
<code>Monitor</code> , <code>TickSnapshot</code>	Tabela por tick, Gantt e estatísticas; o registro de cada tick.
<code>LivePanel</code>	Modo apresentação: redesenha a tela a cada tick.
<code>ConfigLoader</code>	Lê e valida o arquivo de configuração.

Como o executor não guarda estado e todo o contexto está no PCB, trocar de processo é apenas trocar o PCB usado no próximo tick — por isso a troca de contexto custa zero.

11. Testes e validação

- **Gabaritos do enunciado** (cenários A, B e C): conferidos tick a tick — CPU, instrução, acc, filas, bloqueados, eventos, log e estatísticas.
- **Erros de sintaxe:** arquivos com erros propositais; cada erro é informado com a linha correta. `programas/erro1.asm` é um exemplo (label inexistente na linha 6).
- **Caso 1** (`configs/caso1.txt`): lê `-7` pela `SYSCALL 2` e exercita `BRNEG` , `BRZERO` , `BRANY` , `MULT` e `DIV` com negativos ($21 \div -2 = -10$; $-7 \div 2 = -3$). Resultado: término 31, CPU 19, I/O 12.
- **Caso 2** (`configs/caso2.txt`): três processos; dois de mesma prioridade mostram o desempate FIFO na Fila 1, e o terceiro, ao voltar de I/O, interrompe duas vezes um processo da Fila 1, que volta ao topo do grupo com o quantum restante.
- **Fila 1:** `java -cp bin tests.QueueTest` confere a ordem de saída por prioridade e o retorno ao topo do grupo.
- **Portabilidade:** a saída das cinco configurações é idêntica, linha a linha, no Windows e no Linux, com JDK 8 e 21.

Os resultados esperados dos casos 1 e 2, calculados à mão, estão nos comentários dos respectivos arquivos de configuração.

12. Limitações conhecidas

- Um comentário precisa de espaço depois do `#`: `#comentario` é lido como operando e gera erro de sintaxe.
- O arquivo de configuração não aceita comentário no fim de uma linha de processo, porque o caminho do programa pode conter espaços.
- Nomes de variáveis e labels não aceitam letras acentuadas.
- O modo apresentação precisa de um terminal interativo; no Git Bash do Windows a tela não é limpa entre os ticks.